RunnableConfig

RunnableConfig is a TypedDict (A typed dictionary) that defines the standard configuration object passed through every component in the LangChain/LangGraph ecosystem.

```
from langchain_core.runnables import RunnableConfig
```

So basically, it is essentially a structured dict that carries runtime settings alongside any invocation.

The structure

```
class RunnableConfig(TypedDict, total=False):
    tags: List[str]
    metadata: Dict[str, Any]
    callbacks: Callbacks
    run_name: str
    max_concurrency: Optional[int]
    recursion_limit: int
    configurable: Dict[str, Any]   # ← the open slot for user values
    run_id: UUID
```

Note: every field is Optional

the most used field is `configurable`

Example:

```
config = RunnableConfig(
    tags=["production", "sentinel-khmer"],
    metadata={"user_id": "kh-user-001", "region": "Phnom Penh"},
    run_name="scam-detection-run",
    recursion_limit=10,
    configurable={
        "thread_id": "session-abc123",
        "threat_level": "high",
        "language": "khmer"
    }
)

result = graph.invoke({"message": "suspicious QR code detected"}, config=config)
```

Two layer of the `RunnableConfig`

```
RunnableConfig
├── Layer 1: LangChain's own settings
│   ├── tags
│   ├── metadata
│   ├── callbacks
│   ├── run_name
│   ├── recursion_limit
│   ├── max_concurrency
│   └── run_id
│
└── Layer 2: the custom runtime values
    └── configurable  ← open slot for anything we need
            ├── thread_id
            ├── user_id
            └── anything else...
```

- `tags`:

```
config = {"tags": ["production", "v2", "sentinel-khmer"]}
```

- `metadata`

```
config = {"metadata": {"user_id": "kh-001", "region": "Phnom Penh"}}
```

- `callbacks`

```
from langchain_core.callbacks import StdOutCallbackHandler

config = {"callbacks": [StdOutCallbackHandler()]}
```

- `run_name`

```
config = {"run_name": "scam-detection-pipeline"}
```

- `recursion_limit`

```
config = {"recursion_limit": 10}  # default is 25
```

= the maximum of graph steps allowed before LangGraph raises a error **"GraphRecursionError"**, preventing the infinite loops in the agentic workflows.

- `max_concurrency`

```
config = {"max_concurrency": 5}
```

it is only applied when we use the `.batch()`. It controls how many inputs are processed in parallel at the same time.

- `run_id`

```
import uuid
config = {"run_id": uuid.uuid4?()}
```

- `configurable`

```
config = {"configurable": {"thead_id": "session-ABC1", "language": "khmer",
"threat_level": "high"}}
```

How the config propagate under the hood

```
graph.invoke(input, config=config)

# LangChain internally does this for you:
# node_1 receives config
# node_1 calls an LLM → LLM receives config
# node_1 calls a tool → tool receives config
# node_2 receives config
# node_2 calls a retriever → retriever receives config
```

Example:

```
from langchain_core.runnables import RunnableConfig
from langgraph.checkpoint.memory import InMemorySaver
from langgraph.graph import StateGraph, END
from typing import TypedDict

# Define state
class AgentState(TypedDict):
    message: str
    result: str

# Define a node
def analyze_node(state: AgentState, config: RunnableConfig):
    language = config["configurable"].get("language", "en")
    threat   = config["configurable"].get("threat_level", "low")

    return {
        "result": f"Analyzed in {language}, threat={threat}: {state['message']}"
    }
```

```
# Build graph
graph = StateGraph(AgentState)
graph.add_node("analyze", analyze_node)
graph.set_entry_point("analyze")
graph.set_finish_point("analyze")

checkpointer = InMemorySaver()
app = graph.compile(checkpointer=checkpointer)

# Invoke with full RunnableConfig
config: RunnableConfig = {
    "tags": ["production"],
    "metadata": {"user_id": "kh-001"},
    "run_name": "sentinel-khmer-detection",
    "recursion_limit": 15,
    "configurable": {
        "thread_id": "session-001",
        "language": "khmer",
        "threat_level": "high"
    }
}

result = app.invoke({"message": "suspicious QR code"}, config=config)
print(result)
# {"message": "suspicious QR code", "result": "Analyzed in khmer, threat=high:
suspicious QR code"}
```